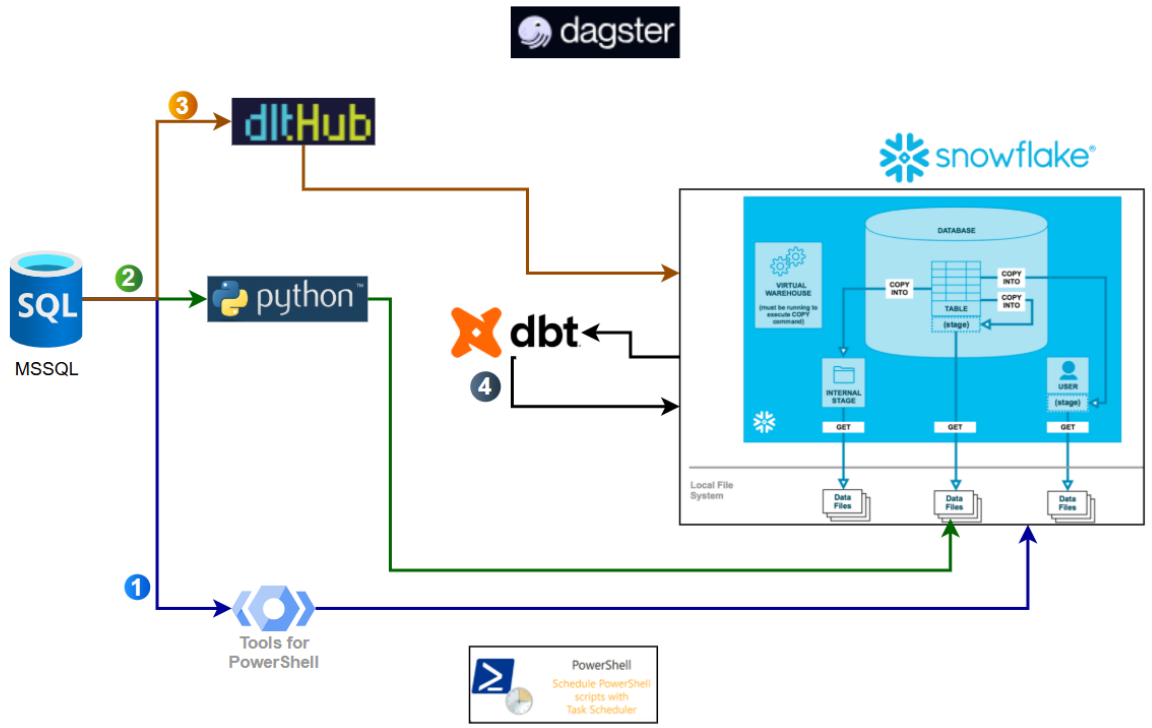
I. Export csv des tables SQL Server vers snowflake (POC) à automatiser une fois par jour (Inventory & Parts) + (Know your customer) (DAI-940)

Ce POC permet de rendre disponibles les données des vues utilisées pour alimenter les agents IA. En effet, ces données, disponibles dans mssql, ont été chargées dans Snowflake en les exportant d'abord en csv. Par conséquent, l'objectif de ce POC est de rendre disponibles ces mêmes données mais grâce à un pipeline automatisé avec des jobs s'exécutant tous les 24h. La solution adoptée a été de mettre en place un processus ELT (Extract Load Transform) de Mssql vers Snowflake.

Ce workflow existe en deux version la première utilise une extraction avec DLT (Data Load Tool) et le second combine BCP (Bulk Copy Program utility) et COPY INTO.



La première version a été optimisé en mettant en œuvre une exécution parallélisée. Cependant, on note une certaine latence lors du chargement des ressources volumineuses. Ainsi, la seconde méthode permet de relever ce défi en optimisant d'avantage le processus de chargement. En effet, dans une première phase elle extrait les données en batch dans un fichier plat puis les dépose (en lots) dans un stage de Snowflake avant de charger les données dans la destination. Cette méthodologie permet de minimiser les temps d'exécution. Pour l'implémentation de cette méthode, deux solutions ont été adoptées. La première utilise un script PowerShell qui fait appel aux outils bcp.exe et snowsql.exe. La deuxième solution a été réalisée en Python. Elle utilise

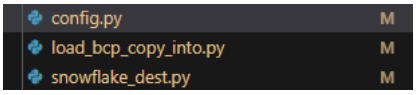
l'exécution de sous processus pour effectuer un bulk copy pour l'extraction des données de SQL server vers un fichier plat. Ensuite, elle charge successivement le fichier dans un stage à l'aide d'un format de fichier et les données dans la table finale en exécutant des requêtes PUT et COPY INTO.

Pour la planification et l'observabilité des solutions, nous avons utilisé le Planificateur de tâches de Windows pour la version avec PowerShell et Dagster pour la version avec Python. Par, ailleurs, cette dernière version a été augmentée d'un projet DBT pour la transformation des données brutes et mettre en place une architecture data mesh (testé avec les uses cases des Mines).

Additionnellement, on note de meilleures performances avec la version utilisant PowerShell.

La mise en œuvre de la version PowerShell nécessite d'installer les outils bcp (avec ODBC) et snowsql puis d'indiquer le chemin de l'installation de bcp.

Les trois fichiers nécessaires pour la version DLT + Dagster sont les suivants :



1. Le fichier config contient une classe **config.py** pour paramétrer mssql et snowflake ainsi que la fonction exporter qui réalise le BCP. Elle définit aussi une méthode d'extraction du schéma de la table (ou vue) mssql puis crée le DDL pour la table Snowflake.
2. Le fichier **snowflake_dest.py** implémente la création du file format, du stage et de la table target vie la méthode **setup_snowflake**. Ensuite elle charge les données via la méthode **upload_to_snowflake.**
3. Le fichier **load_bcp_copy_into.py** implémente la dernière méthode pour réaliser l'ensemble de ces tâches rendant leur appel transparent. Cependant, tous les logs de ces fonctions sont visibles lors de l'exécution pour un meilleurs suivi et débogage.

Pour créer un asset dagster utiliser la fonction **extract_mssql_data(**

```
    snowflake_database = "NEEMBA",
    snowflake_schema = "MINES",
    mssql_table_name = "ref_vw_nba_machines",
    snowflake_table_name = "a_bronze_ref_vw_nba_machines",
    logger = context.log,
)
```

Elle peut être importée de la manière suivante

from mdp_mdp_common.mssql.load_bcp_copy_into import Config, extract_mssql_data

Les paramètres **database** et **schéma** peuvent être omis lorsqu'ils sont renseignés dans le fichier **.env**

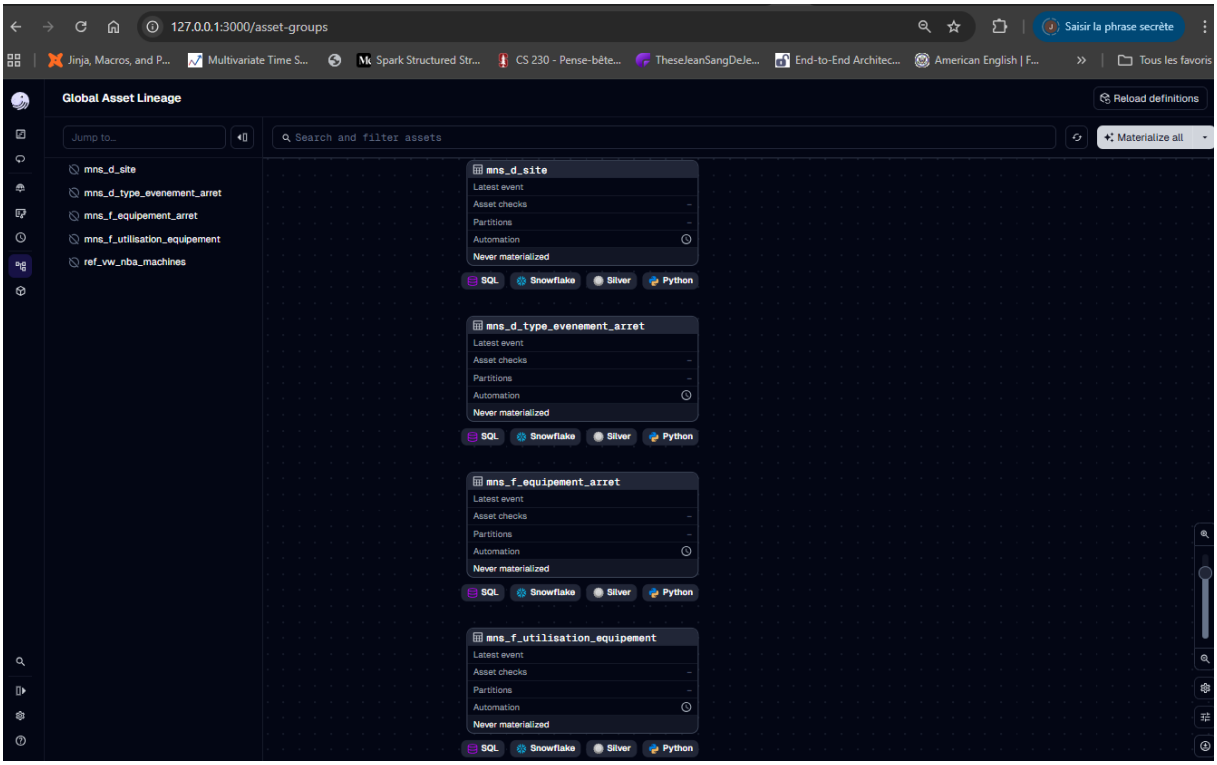
Pour ajouter le BCP dans le dockerfile, on pourra ce servir des commandes ci-après :

```
# BCP executable path

#### Ajouter bcp (Ubuntu/Debian)

curl https://packages.microsoft.com/keys/microsoft.asc | sudo apt-key add -

curl https://packages.microsoft.com/config/ubuntu/$(lsb_release -rs)/prod.list | sudo tee /etc/apt/sources.list.d/mssql-release.list

sudo apt-get update

sudo ACCEPT_EULA=Y apt-get install -y mssql-tools unixodbc-dev

#### Ajouter au PATH

echo 'export PATH="$PATH:/opt/mssql-tools/bin"' >> ~/.bashrc

source ~/.bashrc
```

Lien GitHub avec projet (onglet projet) : click me

Jean Gane Sarr, PhD
Jean Gane Sarr's Credentials wallet • Snowflake

```python
@dg.asset(
    name="ref_vw_nba_machines",
    group_name="data_for_mine",
    description="Ref_vw_nba_machines from MSSQL → Snowflake via BCP + COPY
INTO",
    kinds={"snowflake", "python", "sql", "silver"}
)
def ref_vw_nba_machines_assets(context: dg.AssetExecutionContext) ->
dg.MaterializeResult:
    """ref_vw_nba_machines from MSSQL"""
    result = extract_mssql_data(
        snowflake_database = "NEEMBA",
        snowflake_schema = "MINES",
        mssql_table_name = "ref_vw_nba_machines",
        snowflake_table_name = "a_bronze_ref_vw_nba_machines",
        logger = context.log,
    )

    return dg.MaterializeResult(
        metadata={
            "rows_loaded": dg.MetadataValue.int(result["rows_loaded"]),
        }
    )
```



Jean Gane Sarr, PhD
[Jean Gane Sarr's Credentials wallet • Snowflake]

127.0.0.1:3000/runs/b6c230a3-be70-49c3-a023-c85956a2430c

Aragon | Spark Structured Str... | Django Tutorial Part... | Trouver un cours en... | TheseJeanSangDe

Runs / b6c230a3 | Success | v_Inventory_Parts_Ops | 26 févr., 11:33:24 | 0:01:28

Hide not started steps

Events | stdout | stderr | Filter... | Levels (5/6)

TIMESTAMP	OP	EVENT TYPE	INFO
			==========
11:33:40,466	v_Inventory_Parts_Ops	INFO	PIPELINE MSSQL → SNOWFLAKE
11:33:40,484	v_Inventory_Parts_Ops	INFO	Méthode: BCP + COPY INTO
11:33:40,502	v_Inventory_Parts_Ops	INFO	==========
11:33:40,521	v_Inventory_Parts_Ops	INFO	==========
11:33:40,538	v_Inventory_Parts_Ops	INFO	Export BCP depuis SQL Server pour la table V_Inventory_Parts_Ops
11:33:40,555	v_Inventory_Parts_Ops	INFO	==========
11:33:40,612	v_Inventory_Parts_Ops	INFO	Fichier existant supprimé: /tmp/mssql_export.csv
11:33:40,627	v_Inventory_Parts_Ops	INFO	Exécution BCP...
11:34:32,504	v_Inventory_Parts_Ops	INFO	Export BCP terminé en 51.86s
11:34:32,525	v_Inventory_Parts_Ops	INFO	Temps BCP: 51.86s
11:34:32,541	v_Inventory_Parts_Ops	INFO	Fichier: /tmp/mssql_export.csv
11:34:32,556	v_Inventory_Parts_Ops	INFO	Taille: 32.31 MB
11:34:32,572	v_Inventory_Parts_Ops	INFO	==========
11:34:32,588	v_Inventory_Parts_Ops	INFO	Setup Snowflake
11:34:32,604	v_Inventory_Parts_Ops	INFO	==========
11:34:35,681	v_Inventory_Parts_Ops	INFO	Création du file format CSV...
11:34:35,830	v_Inventory_Parts_Ops	INFO	File format mssql_csv_file_format créé
11:34:35,850	v_Inventory_Parts_Ops	INFO	Création du stage...
11:34:36,090	v_Inventory_Parts_Ops	INFO	Stage MSSQL_DIRECT_STAGE créé
11:34:36,107	v_Inventory_Parts_Ops	INFO	Création de la table NEEMBA.EQUIPEMENT.V_Inventory_Parts_Ops...
11:34:38,074	v_Inventory_Parts_Ops	INFO	Table V_Inventory_Parts_Ops créée avec succès
11:34:38,097	v_Inventory_Parts_Ops	INFO	Localisation: NEEMBA.EQUIPEMENT.V_Inventory_Parts_Ops
11:34:38,374	v_Inventory_Parts_Ops	INFO	==========
11:34:38,389	v_Inventory_Parts_Ops	INFO	Upload vers Snowflake Stage
11:34:38,413	v_Inventory_Parts_Ops	INFO	==========
11:34:40,895	v_Inventory_Parts_Ops	INFO	Upload de mssql_export.csv...
11:34:44,437	v_Inventory_Parts_Ops	INFO	Upload terminé en 3.52s
11:34:44,631	v_Inventory_Parts_Ops	INFO	Fichiers dans le stage: 1
11:34:45,348	v_Inventory_Parts_Ops	INFO	==========
11:34:45,367	v_Inventory_Parts_Ops	INFO	COPY INTO Snowflake
11:34:45,380	v_Inventory_Parts_Ops	INFO	==========
11:34:47,884	v_Inventory_Parts_Ops	INFO	Chargement dans AI_V_INVENTORY_PARTS_OPS...
11:34:51,860	v_Inventory_Parts_Ops	INFO	✓ mssql_direct_stage/mssql_export.csv.gz: 209,131 lignes
11:34:51,881	v_Inventory_Parts_Ops	INFO	COPY INTO terminé en 3.96s
11:34:51,900	v_Inventory_Parts_Ops	INFO	Nombre de lignes chargées : 209,131
11:34:51,927	v_Inventory_Parts_Ops	INFO	Nombre d'erreurs : 1
11:34:52,087	v_Inventory_Parts_Ops	INFO	Total dans la table AI_V_INVENTORY_PARTS_OPS: 209,130
11:34:52,378	v_Inventory_Parts_Ops	INFO	
			==========
11:34:52,392	v_Inventory_Parts_Ops	INFO	PIPELINE TERMINÉ AVEC SUCCÈS
11:34:52,408	v_Inventory_Parts_Ops	INFO	Temps total Snowflake: 71.93s
11:34:52,422	v_Inventory_Parts_Ops	INFO	Total lignes insérées: 209,131
11:34:52,437	v_Inventory_Parts_Ops	INFO	Débit: 2907 rows/sec
11:34:52,456	v_Inventory_Parts_Ops	INFO	==========
11:34:52,476	v_Inventory_Parts_Ops	STEP_OUTPUT	Yielded output "result" of type "Any". (Type check passed).
			rows_loaded 209131

Jean Gane Sarr, PhD
Jean Gane Sarr's Credentials wallet • Snowflake

```
Sans titre1.ps1 | load_mssql_to_snowflake_ps.ps1 | load_vm_devis_dashboard_am.ps1 ✕ | Sans titre3.ps1* | Sans titre4.ps1*
211      "-d", $SnowflakeDatabase,
212      "-s", $SnowflakeSchema,
213      "-q", $sql_copy_into
214  )
215  try {
216       Write-Host "Executing Snowflake COPY INTO command..."
217       & snowsql @snowsqlArgs
218       Write-Host "✅ Data insert to the target table $SnowflakeTable." -ForegroundColor Green
219  }
220  catch {
221      Write-Host "❌ Error executing the put command: $_" -ForegroundColor Red
222      throw
223  }
```

```
Executing Snowflake PUT command...
* SnowSQL * v1.4.5
Type SQL statements or !help
+-----------------------------+-------------------------------+-------------+-------------+--------------------+--------------------+----------+---------+
| source                      | target                        | source_size | target_size | source_compression | target_compression | status   | message |
|-----------------------------+-------------------------------+-------------+-------------+--------------------+--------------------+----------+---------|
| mssql_vlnk_exported_data.csv | mssql_vlnk_exported_data.csv.gz |  1349787031 |   217970800 | NONE               | GZIP               | UPLOADED |         |
+-----------------------------+-------------------------------+-------------+-------------+--------------------+--------------------+----------+---------+
1 Row(s) produced. Time Elapsed: 193.833s
Goodbye!
✅ Data put to MSSQL_DIRECT_STAGE
Executing Snowflake COPY INTO command...
* SnowSQL * v1.4.5
Type SQL statements or !help
+-------------------------------------------------+--------+-------------+-------------+-------------+-------------+-------------+------------------+-----
----------------+-------------------------+
| file                                            | status | rows_parsed | rows_loaded | error_limit | errors_seen | first_error | first_error_line | firs
error_character | first_error_column_name |
|-------------------------------------------------+--------+-------------+-------------+-------------+-------------+-------------+------------------+-----
----------------+-------------------------|
| mssql_direct_stage/mssql_vlnk_exported_data.csv.gz | LOADED |     9999999 |     9999999 |           1 |           0 | NULL        |             NULL |
          NULL | NULL                    |
+-------------------------------------------------+--------+-------------+-------------+-------------+-------------+-------------+------------------+-----
----------------+-------------------------+
1 Row(s) produced. Time Elapsed: 46.409s
Goodbye!
✅ Data insert to the target table a_bronze_vlinklocalisation_.
Data loaded into Snowflake table a_bronze_vlinklocalisation_

PS C:\WINDOWS\system32>
```

II. Mise à jour des fichiers et rapports de la location

Il existe un document de la procédure à suivre dans le dossier Location : Pape Amadou et Christophe.

II. Envoie des rapports Dashboard logistique Service et Service Data

Ndeye Signara et Abdoulaye